ProctorAI AI Based Online Proctoring System

***TeamID: [Write Your Assigned Team ID Here]**

Student Name 1 (Roll No: XXXXXXXX)
Student Name 2 (Roll No: XXXXXXXX)
Student Name 3 (Roll No: XXXXXXXX)
Student Name 4 (Roll No: XXXXXXXX)
Student Name 5 (Roll No: XXXXXXXX)
Student Name 6 (Roll No: XXXXXXXX)
Department of Computer Science Engineering and Technology
Bennett University, India

GitHub Link: https://github.com/MehulMTyagi/PROCTORAI

Abstract - ProctorAI is an AI based online proctoring system designed for college level remote assessments. The problem addressed by the project is that online tests are difficult to supervise because a candidate may use another device, leave the camera frame, receive help, switch applications, or stop monitoring permissions. The proposed system combines a candidate portal, examiner/proctor portal, FastAPI backend, local AI sidecar, browser integrity monitoring, and protected evidence handling. The monitoring design includes face absence detection, multiple person detection, YOLO based phone detection, liveness verification, screen recording checks, copy and paste tracking, fullscreen and focus-loss events, and a planned identity identification step using registered and live candidate face images. Instead of automatically declaring misconduct, the system sends evidence backed integrity events to the proctor for review. The current Phase 2 work focuses on architecture, module integration, dataset planning, and evaluation design; final quantitative results will be updated after complete testing before project submission.

Index Terms - Online proctoring, computer vision, YOLO, face detection, liveness verification, identity identification, integrity events, evidence review.

I. INTRODUCTION

Online examinations are useful because they allow students to appear for tests without being physically present in a classroom. At the same time, they create a serious supervision problem. In a normal examination hall, an invigilator can observe the candidate, check the surroundings, and notice suspicious actions. In an online test, the candidate may use a phone, receive help from another person, leave the camera frame, switch tabs, copy answers, or stop screen recording.

The problem addressed in this project is the lack of reliable and reviewable supervision in online assessments. Many simple online test systems only provide questions and answer submission. They do not provide strong setup checks, candidate identity verification, browser activity tracking, or evidence based proctor review. This makes them weak for serious college examinations.

ProctorAI is proposed as an AI based online proctoring system for college assessments. It combines a candidate portal, examiner/proctor portal, backend API, local AI sidecar, screen recording checks, camera and microphone monitoring, liveness verification, and event evidence review. The system is designed to detect suspicious behavior and send integrity events to a proctor instead of automatically punishing the candidate.

The main contribution of this project is a privacy-aware multi-signal proctoring workflow. The system uses AI and browser signals together: face absence, multiple persons, phone detection, copy and paste, focus lost, fullscreen exit, screen recording stopped, background audio, liveness failure, and identity identification. The proctor receives proof and can review the event before making a decision.

The motivation for including identity identification is that a candidate may complete setup checks but another person may attempt the assessment. The project therefore includes an identity workflow where a live candidate image is compared with a registered candidate image. This feature is planned as a reviewable verification step rather than an automatic rejection mechanism, because face recognition systems can be affected by camera quality, lighting, pose, and demographic bias.

The scope of Phase 2 is to present the architecture, modules, research basis, dataset plan, and evaluation strategy. The project will be completed and fully tested before final project submission, but this report avoids reporting final accuracy values before the validation set is complete.

II. LITERATURE SURVEY

AI based proctoring research shows that online examination monitoring usually combines several signals rather than relying on one detector. A systematic review of AI based proctoring systems reported common use of face monitoring, gaze or behavior analysis, audio analysis, and log based activity tracking [1]. This supports the direction of ProctorAI because cheating or suspicious behavior can appear through the camera, browser, microphone, or device state.

Face recognition and face detection are major components of proctoring systems. Surveys on face recognition based exam monitoring discuss methods such as Viola-Jones, CNN based face detectors, and MTCNN [2], [9]. For ProctorAI, face detection is used first as a presence and continuity signal: the system checks whether the candidate is visible, whether multiple people appear, and whether identity verification should be repeated.

Object detection is required for detecting mobile phones and additional persons. YOLO style detectors are widely used for real time detection because they process an image in a single forward pass [5], [6]. COCO is useful as a base object detection dataset because it provides common object categories in natural scenes [7]. However, general object datasets do not fully represent exam room conditions, so a small custom validation set is still needed.

Identity identification and liveness verification are also important for a production grade proctoring system. FaceNet and ArcFace show how face embeddings can be used to compare whether two face images belong to the same person [11], [12]. Face anti-spoofing surveys show that liveness checks are needed because a photo, video

replay, or mask can attack a face recognition system [13]. ProctorAI therefore treats identity verification and liveness as separate steps.

Recent discussions on remote proctoring also warn about bias, privacy, and unfair automatic decisions [15]. This is why ProctorAI uses human review: the AI system raises an integrity event with proof, but the final decision remains with the proctor.

Study Area	Main Method	Limitation Observed	Use in ProctorAI
AI proctoring surveys	Review of face, audio, browser, and behavior signals	Many systems depend heavily on automated flags	Use multi-signal monitoring with human review
Face recognition proctoring	Face detection and verification models	Identity alone cannot detect browser misuse	Combine identity with browser and screen events
YOLO object detection	Real-time object detector for common objects	General datasets may not match exam rooms	Use YOLO plus custom validation scenarios
Face anti-spoofing	Liveness and presentation attack detection	Spoof attacks vary across devices and lighting	Use active liveness before test start
Remote proctoring fairness	Bias and vulnerability analysis	Automated decisions can be unfair	Keep proctor decision separate from AI risk score

Table I. Summary of literature gaps and ProctorAI design response.

The literature gap is that many approaches focus strongly on one signal, such as face recognition or object detection, while practical online cheating can happen through several channels at the same time. A candidate may remain visible on camera but copy content from another source, or may avoid browser misuse but use a second device outside the screen. ProctorAI addresses this gap by combining setup verification, browser activity, camera events, audio events, identity checks, and evidence review in one workflow.

III. PROPOSED METHODOLOGY

A. System Architecture

The proposed methodology follows a multi-module architecture. The candidate first selects an assigned assessment and completes setup checks. These checks include consent, backend connectivity, camera permission, microphone permission, screen recording, connected display audit, liveness verification, and identity identification. The identity module compares a registered candidate image with a live setup image and sends uncertain cases to the proctor for review.

During the test, the candidate portal monitors browser and media signals. Browser integrity events include copy, paste, fullscreen exit, focus lost, suspicious shortcuts, and stopped screen recording. Camera based events include face absent, multiple persons, phone detected, and liveness failure. Audio events include microphone interruption and background voice or noise signals. Evidence is attached as short clips or proof metadata instead of continuously streaming all media.

The threat model considered in this project includes impersonation, use of a mobile phone, another person helping the candidate, leaving the camera frame, using copy-paste or shortcut keys, switching away from the test window, stopping screen recording, and attempting to use an extra display. These risks were selected because they are realistic for a college online assessment and can be detected using browser APIs, camera analysis, microphone analysis, and setup checks.

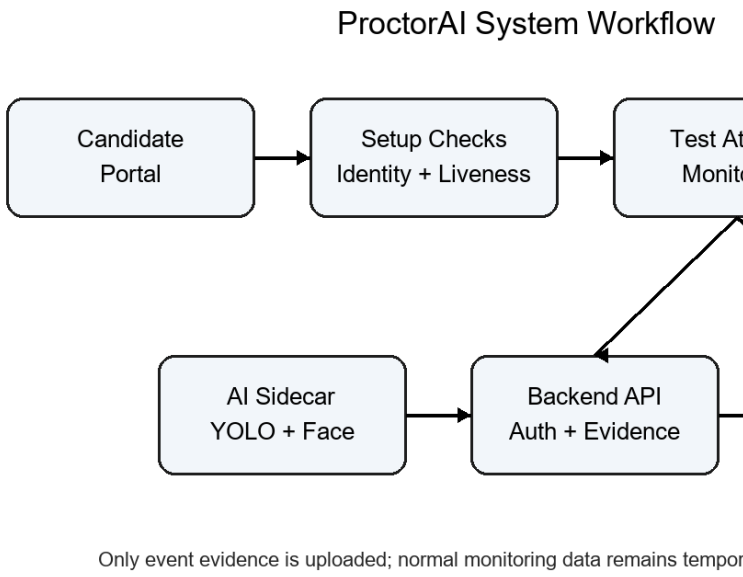


Fig. 1. Proposed ProctorAI system workflow.

The backend stores users, assessments, attempts, answers, integrity events, evidence metadata, and proctor review decisions. A WebSocket or equivalent real time channel sends new integrity events to the proctor portal. The proctor portal displays each event with event type, severity, confidence, timestamp, evidence status, and review action. The final system will support confirmed suspicious, false positive, and inconclusive decisions.

The AI sidecar is responsible for local computer vision inference. YOLO is used for phone and object detection, while face analysis is used for face count, face absence, liveness support, and identity continuity. This architecture keeps most normal monitoring data local and uploads only event specific evidence, which is more privacy aware than continuous cloud recording.

B. Identity Identification Workflow

The identity identification feature is designed as a setup step before the test begins. The candidate provides or captures a registered face image and then captures a live selfie during setup. The system extracts a face representation from both images and compares them using a similarity threshold. If the similarity is above the accepted threshold, the identity check passes. If the score is close to the threshold or below it, the proctor receives an identity review event instead of the candidate being automatically rejected.

This design separates identity verification from liveness. Identity verification answers the question: is this the expected candidate? Liveness verification answers a different question: is the candidate physically present rather than showing a static photo or replayed video? Keeping these checks separate makes the workflow easier to explain and easier for a proctor to review.

C. Integrity Event Generation

Integrity events are generated only when a monitored condition crosses a threshold. For example, face absence should not be triggered by a single weak frame, because lighting and camera focus may temporarily affect detection. Instead, the system uses debounce rules such as consecutive-frame confirmation or confidence

thresholds. Browser events such as copy, paste, fullscreen exit, and focus lost are generated directly from browser event listeners because they are discrete actions.

Each event stores event type, timestamp, candidate session, severity, confidence or rule metadata, and evidence status. Camera based events use camera proof, while browser activity events should use screen evidence. This difference is important because a camera clip cannot prove a copy-paste action, but a short screen clip can provide better context for the proctor.

Step	Process	Output
1	Candidate logs in and selects assigned test	Assessment session is initialized
2	Setup checks request consent, camera, microphone, screen, liveness, and identity	Candidate is allowed or blocked before test
3	Candidate starts assessment after required checks pass	Attempt record and heartbeat stream begin
4	Browser, camera, audio, and AI sidecar monitor signals	Raw signals are converted into event candidates
5	Debounce and threshold rules confirm event candidates	Integrity event is created
6	Evidence clip or proof metadata is attached	Event becomes reviewable
7	Proctor portal receives live update	Human review decision is recorded

Table II. Stepwise processing flow for ProctorAI.

Module	Purpose	Phase 2 Status
Candidate portal	Assignment selection, setup checks, test attempt	Implemented and being refined
Proctor portal	Live event queue and evidence review	Implemented with review workflow planned for final hardening
AI sidecar	Face, liveness, and phone/object analysis	Implemented for core signals
Identity identification	Compare registered and live candidate identity	Included in final feature scope
Evidence handling	Store proof clips and hashes for review	Implemented for event evidence

Table III. Major ProctorAI modules and Phase 2 status.

D. Privacy and Review Policy

The system is intentionally designed as a review-support system rather than an automatic punishment system. Normal monitoring data remains temporary wherever possible. Evidence is attached only when an integrity event is generated. The proctor can mark an event as confirmed suspicious, false positive, or inconclusive. This is useful for academic fairness because AI models can make mistakes in low light, poor camera angles, noisy rooms, or unstable network conditions.

The review policy also separates assessment score from integrity review. A candidate may complete the test and receive an academic score, while the proctor separately reviews integrity events. This separation is important because a suspicious event does not always prove misconduct. For example, a short face absence could happen due to camera freeze, and background voice could come from a family member in a shared room. The proctor review layer helps prevent unfair automatic decisions.

The evidence policy follows a minimal collection approach. Continuous live streaming is avoided in the planned design unless specifically enabled for a future deployment. Instead, the system stores short event-related clips, hashes, timestamps, and metadata. This makes the system more suitable for a college project where privacy and storage cost both matter.

IV. EXPERIMENTAL RESULTS

A. Dataset Description

The project uses a combination of public datasets, pretrained model knowledge, and custom validation data. COCO is selected for object detection support because it contains common objects such as persons and mobile phones in natural contexts [7]. WIDER FACE is relevant for face detection benchmarking because it contains many face images with variation in scale, pose, and occlusion [8]. A custom validation set will be prepared for exam-like cases such as normal candidate sitting, face absent, multiple persons, phone visible, poor lighting, background movement, and screen recording interruption.

Data Source	Used For	Sample Type	Ethical Note
COCO	Phone/person/object reference	Public images	Used as model reference only
WIDER FACE	Face detection reference	Public face images	Used for understanding detection variation
Custom setup images	Exam-like validation	Team-created scenarios	Use consent and avoid unnecessary personal storage
Browser event logs	Copy/paste/focus validation	Synthetic event actions	No private content stored
Evidence clips	Review workflow testing	Short event clips	Store only event-related proof

Table IV. Dataset and validation data plan.

B. Evaluation Metrics

For event detection, the planned evaluation metrics are precision, recall, F1-score, false positive rate, event delivery latency, and evidence availability rate. Precision measures how many generated events are actually relevant. Recall measures how many expected events are detected. F1-score balances precision and recall. Latency measures how quickly the proctor receives an event after it occurs. Evidence availability rate measures whether each event includes usable proof for review.

The formulas used for classification-style event evaluation are: Precision = TP / (TP + FP), Recall = TP / (TP + FN), and F1-score = 2 x Precision x Recall / (Precision + Recall). Here TP means a suspicious event was correctly detected, FP means the system raised an unnecessary event, and FN means a suspicious event was missed. These metrics will be computed separately for each signal type because phone detection, face absence, copy-paste, and identity mismatch have different error patterns.

Metric	Meaning	Why It Matters	Target

			Direction
Precision	Correct events among all raised events	Reduces unnecessary proctor workload	Higher is better
Recall	Detected events among all expected events	Reduces missed suspicious behavior	Higher is better
F1-score	Balance of precision and recall	Useful when both false alarms and missed events matter	Higher is better
Latency	Time from event occurrence to proctor view	Important for real-time review	Lower is better
Evidence availability	Events with playable proof	Allows fair human review	Higher is better

Table V. Evaluation metrics planned for ProctorAI.

C. Current Validation and Comparison

Signal	Detector or Source	Expected Output	Evidence Type	Status
Face absent	Camera and face count	FACE_ABSENT event	Camera clip/proof	Implemented
Multiple persons	Camera and face count	MULTIPLE_PERSONS event	Camera clip/proof	Implemented
Phone visible	YOLO object detection	PHONE_DETECTED event	Camera clip/proof	Implemented
Copy or paste	Browser event listener	Clipboard event	Screen clip	Implemented
Identity mismatch	Face comparison	Identity review event	Planned identity proof	Final scope

Table VI. Planned and current integrity signal validation.

At this stage, the report records Phase 2 design and module validation rather than final model accuracy. Final numerical performance values will be added after full testing on the custom validation set. This is academically safer than reporting artificial accuracy values before the complete experiment is finished.

D. Baseline Comparison Plan

Method	Camera Events	Browser Events	Evidence Review	Expected Limitation
Manual online invigilation	Human observation	Limited unless screen shared	Manual notes	Difficult to scale
Camera-only proctoring	Face and object events	Not covered	Camera proof only	Cannot prove browser misuse
Browser-only monitoring	Not covered	Focus, clipboard, fullscreen	Screen proof	Cannot detect phones or extra persons
ProctorAI	Face, phone, liveness, identity	Clipboard, focus, fullscreen, screen stop	Camera and screen evidence	Needs careful threshold tuning

Table VII. Comparison plan with baseline proctoring approaches.

E. Hyperparameter Tuning

The main parameters to tune are object detection confidence threshold, consecutive-frame count for event confirmation, liveness motion threshold, audio activity threshold, and identity match threshold. The tuning strategy will compare lenient, moderate, and strict settings on the validation set. The selected configuration should reduce false positives while still detecting important integrity risks.

Parameter	Candidate Values	Validation Signal	Selection Rule
YOLO confidence	0.25, 0.40, 0.60	Phone false positives and misses	Best F1-score
Face absent frames	2, 3, 5 consecutive frames	Temporary camera glitches	Low false positive rate
Liveness threshold	Low, medium, high motion	Challenge pass/fail reliability	Best valid-user pass rate
Audio threshold	Low, medium, high level	Background noise detection	Balanced false alarms
Identity similarity	Lenient, moderate, strict	Selfie match validation	Manual review band near threshold

Table VIII. Hyperparameter tuning plan.

F. Implementation Environment

The implementation uses a React and TypeScript candidate portal, a React based examiner/proctor portal, a FastAPI backend, SQLite for reliable local development, PostgreSQL as the production target, and a Python AI sidecar for computer vision tasks. The local development environment uses fixed service ports for the API, AI engine, web portal, and candidate portal so that candidate-to-proctor event flow can be tested repeatedly.

G. Validation Scenarios

Scenario	Expected Event	Evidence Expected	Pass Condition
Candidate remains visible	No suspicious event	None	No false alert generated
Candidate leaves frame	FACE_ABSENT	Camera proof	Event appears in proctor queue
Phone shown to camera	PHONE_DETECTED	Camera clip	YOLO confidence stored
Copy and paste attempted	Clipboard event	Screen clip	Browser event sent live
Screen sharing stopped	SCREEN_RECORDING_STOPPED	Screen state proof	Test is blocked or flagged
Different	IDENTITY_MISMATCH or	Identity	Proctor

person appears	UNKNOWN_FACE	proof	review requested

Table IX. Validation scenarios for final testing.

The final validation will be performed by running each scenario multiple times under normal light, low light, different camera angles, and temporary network interruptions. This is needed because proctoring systems fail most often in ordinary real-world conditions rather than in clean laboratory conditions. The expected result is not zero false positives, but a controlled event workflow where false positives can be reviewed and dismissed by the proctor.

V. CONCLUSION

This Phase 2 report presents the design and development direction of ProctorAI, an AI based online proctoring system for college assessments. The system addresses the problem of weak supervision in online tests by combining candidate setup checks, identity verification, liveness, object detection, browser integrity monitoring, evidence upload, and proctor review.

The current limitation is that final quantitative accuracy and latency results still need to be measured after completing the full validation dataset and final project testing. Future work before final submission includes completing identity identification, improving hosted deployment reliability, tuning thresholds, and generating final proctoring reports.

REFERENCES

[1] A. Nigam, R. Pasricha, T. Singh, and P. Churi, "A systematic review on AI-based proctoring systems: Past, present and future," Education and Information Technologies, 2021.

[2] R. A. Pramudita et al., "Survey of online exam proctoring model to detect cheating behavior based on face recognition," IEEE Conference Publication, 2022.

[3] A. H. M. et al., "Towards effective and efficient online exam systems using deep learning-based cheating detection approach," Computers and Education: Artificial Intelligence, vol. 3, 2022.

[4] D. Somavarapu et al., "A systematic-narrative review of online proctoring systems and a case for open standards," 2025.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," Proc. CVPR, 2016.

[6] J. Terven and D. Cordova-Esparza, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," arXiv:2304.00501, 2023.

[7] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," Proc. ECCV, 2014.

[8] S. Yang, P. Luo, C. C. Loy, and X. Tang, "WIDER FACE: A face detection benchmark," Proc. CVPR, 2016.

[9] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," IEEE Signal Processing Letters, 2016.

[10] V. Bazarevsky et al., "BlazeFace: Sub-millisecond neural face detection on mobile GPUs," arXiv:1907.05047, 2019.

[11] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," Proc. CVPR, 2015.

[12] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," Proc. CVPR, 2019.

[13] Z. Yu, Y. Qin, X. Li, C. Zhao, Z. Lei, and G. Zhao, "Deep learning for face anti-spoofing: A survey," arXiv:2106.14948, 2021.

[14] T. Baltrusaitis, A. Zadeh, Y. C. Lim, and L.-P. Morency, "OpenFace 2.0: Facial behavior analysis toolkit," Proc. FG, 2018.

[15] M. Burgess et al., "Watching the watchers: Bias and vulnerability in remote proctoring software," USENIX Security Symposium, 2022.